

第十一章 索引与散列

11.1 基本概念

两种基本索引类型：

- **顺序索引**：基于值的顺序排序
- **散列索引**：基于将值平均分布到若干散列桶中，一个值所属的散列桶是由一个函数决定的，这个函数称为**散列函数**

对每种技术的评价基于以下因素：

- **访问类型**(access type)：能有效支持的访问类型。访问类型可以包括找到具有特定属性值的记录，以及找到属性值落在某个特定范围内的记录。
- **访问时间**(access time)：在查询中使用该技术找到一个特定数据项或数据项集所需的时间。
- **插入时间**(insertion time)：插入一个新数据项所需的时间。该值包括找到插入这个新数据项的正确位置所需的时间，以及更新索引结构所需的时间。
- **删除时间**(deletion time)：删除一个数据项所需的时间。该值包括找到待删除项所需的时间，以及更新索引结构所需的时间。
- **空间开销**(space overhead)：索引结构所占用的额外存储空间。倘若存储索引结构的额外空间大小适度，通常牺牲一定的空间代价来换取性能的提高是值得的。

用于在文件中查找记录的属性或属性集称为**搜索码**

11.2 顺序索引

如果文件按照某个搜索码指定的顺序排序，那么该搜索码对应的索引称为**聚集索引**，也称为**主索引**

搜索码制定的顺序与文件中记录的物理顺序不同的索引称为**非聚集索引**或**辅助索引**

索引顺序文件：搜索码上有聚集索引的文件，搜索码通俗的说就是搜索数据库内容时用的，通常就是索引。

example:

10101	Srinivasan	Comp. Sci.	65000		
12121	Wu	Finance	90000		
15151	Mozart	Music	40000		
22222	Einstein	Physics	95000		
32343	El Said	History	60000		
33456	Gold	Physics	87000		
45565	Katz	Comp. Sci.	75000		
58583	Califieri	History	62000		
76543	Singh	Finance	80000		
76766	Crick	Biology	72000		
83821	Brandt	Comp. Sci.	92000		
98345	Kim	Elec. Eng.	80000		



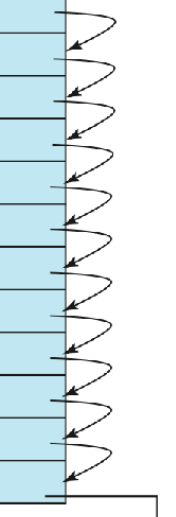
11.2.1 稠密索引和稀疏索引

索引项或索引记录由一个搜索码值和指向具有该搜索码的一条或多条记录的指针构成

其实就是外部的指针，可以快速的找到对应搜索码值的记录。

- **稠密索引**：在稠密索引中，文件中的每个搜索码值都有一个索引项

10101		→	10101	Srinivasan	Comp. Sci.	65000	
12121		→	12121	Wu	Finance	90000	
15151		→	15151	Mozart	Music	40000	
22222		→	22222	Einstein	Physics	95000	
32343		→	32343	El Said	History	60000	
33456		→	33456	Gold	Physics	87000	
45565		→	45565	Katz	Comp. Sci.	75000	
58583		→	58583	Califieri	History	62000	
76543		→	76543	Singh	Finance	80000	
76766		→	76766	Crick	Biology	72000	
83821		→	83821	Brandt	Comp. Sci.	92000	
98345		→	98345	Kim	Elec. Eng.	80000	



- **稀疏索引**：在稀疏索引中，只为搜索码的某些值建立索引项

使用的前提是：关系按照搜索码的顺序存储

查找的时候找到最大的小于等于待查找的码值的搜索码，然后从该位置开始，沿着文件中的指针查找。

Biology		76766	Crick	Biology	72000	
Comp. Sci.		10101	Srinivasan	Comp. Sci.	65000	
Elec. Eng.		45565	Katz	Comp. Sci.	75000	
Finance		83821	Brandt	Comp. Sci.	92000	
History		98345	Kim	Elec. Eng.	80000	
Music		12121	Wu	Finance	90000	
Physics		76543	Singh	Finance	80000	
		32343	El Said	History	60000	
		58583	Califieri	History	62000	
		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		33465	Gold	Physics	87000	

稀疏索引占空间小，维护的

11.2.2 多级索引

具有两级或两级以上的索引称为**多级索引**

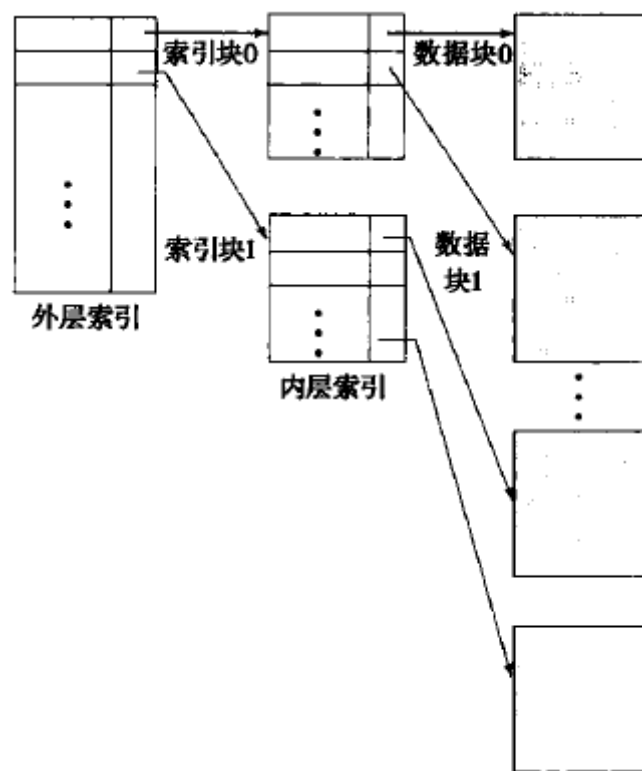


图 11-5 二级稀疏索引

11.2.3 索引的更新

每当文件有记录更新时，索引都需要更新

- **插入。**系统首先用出现在待插入记录中的搜索码值进行查找，并根据索引是稠密索引还是稀疏索引而进行下一个操作。

□ **稠密索引：**

1. 如果该搜索码值不在索引中，系统就在索引中合适的位置插入具有该搜索码值的索引项。
2. 否则进行如下操作：
 - a. 如果索引项存储的是指向具有相同搜索码值的所有记录的指针，系统就在索引项中增加一个指向新记录的指针。
 - b. 否则，索引项存储一个仅指向具有相同搜索码值的第一条记录的指针，系统把待插入的记录放到具有相同搜索码值的其他记录之后。

□ **稀疏索引：**我们假设索引为每个块保存一个索引项。如果系统创建一个新的块，它会将新块中出现的第一个搜索码值(按照搜索码的顺序)插入到索引中。另一方面，如果这条新插入的记录含有块中的最小搜索码值，那么系统就更新指向该块的索引项；否则，系统对索引不做任何改动。

- **删除。**为删除一条记录，系统首先查找要删除的记录，然后下一步的操作取决于索引是稠密索引还是稀疏索引。

□ **稠密索引：**

1. 如果被删除的记录是具有这个特定搜索码值的唯一的一条记录，系统就从索引中删除相应的索引项。
2. 否则采取如下操作：
 - a. 如果索引项存储的是指向所有具有相同搜索码值的记录的指针，系统就从索引项中删除指向被删除记录的指针。
 - b. 否则，索引项存储的是指向具有该搜索码值的第一条记录的指针。在这种情况下，如果被删除的记录是具有该搜索码值的第一条记录，系统就更新索引项，使其指向下一条记录。

□ **稀疏索引：**

1. 如果索引不包含具有被删除记录搜索码值的索引项，则索引不必做任何修改。
2. 否则系统采取如下操作：
 - a. 如果被删除的记录是具有该搜索码值的唯一记录，系统用下一个搜索码值(按搜索码顺序)的索引记录替换相应的索引记录。如果下一个搜索码值已经有一个索引项，则删除而不是替换该索引项。
 - b. 否则，如果该搜索码值的索引记录指向被删除的记录，系统就更新索引项，使其指向具有相同搜索码值的下一条记录。

11.2.4 辅助索引

辅助索引必须是稠密索引，对每个搜索码值都有一个索引项

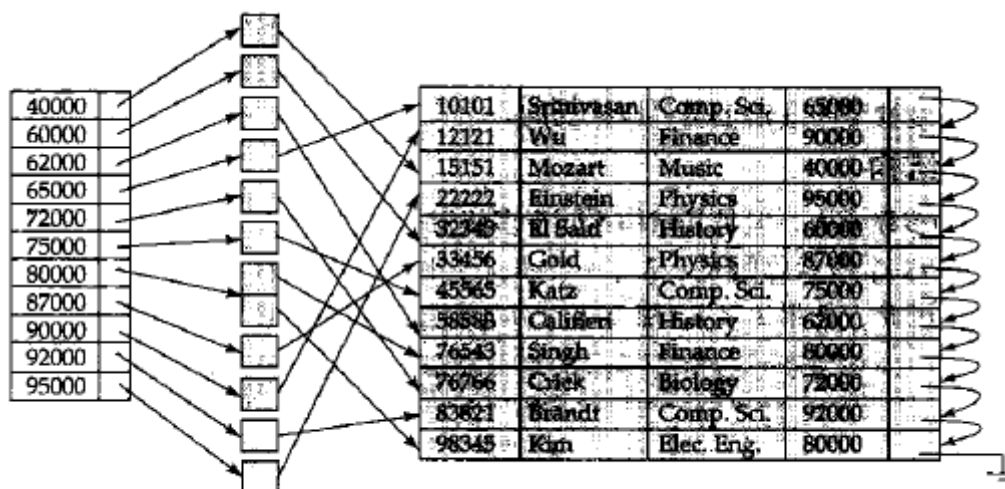


图 11-6 *instructor* 文件的辅助索引，基于非候选码 *salary*

11.2.5 多码上的索引

一个包含多个属性的搜索码被称为**符合搜索码**

11.5 多码访问

11.5.1 使用多个单码索引

```
select  ID
from    instructor
where   dept_name = "Finance" and salary = 80000
```

处理这个查询可以有三种策略：

1. 利用 *dept_name* 上的索引，找出属于金融系的所有记录。检查每条记录是否满足 *salary* = 80 000。
2. 利用 *salary* 上的索引，找出所有工资等于 \$80 000 的记录。检查每条记录是否满足 *dept_name* = "Finance"。
3. 利用 *dept_name* 上的索引找出指向属于金融系的记录的所有指针。同样，利用 *salary* 上的索引找出指向工资等于 \$80 000 的记录的所有指针。计算这两个指针集合的交。交集集中的那些指针指向金融系中工资等于 \$80 000 的记录。

11.5.2 多码索引

在复合的搜索码上建立和使用索引

11.5.3 覆盖索引

覆盖索引存储一些属性（但不是搜索码属性）的值以及指向记录的指针

11.6 静态散列

基于散列技术的文件组织使我们能够避免访问索引结构

桶：表示能存储一条或多条记录的一个存储单位

散列文件组织中，我们通过计算所需记录搜索码值上的一个函数直接获得包含该记录磁盘快地址

散列索引组织中，我们把搜索码以及与它们相关联的指针组织成一个散列文件结构

11.6.1 散列函数

我们希望搜索码值分配到桶中并且具有以下分布特性的散列函数：分布是**均匀的**、**随机的**

11.6.2 桶溢出处理

如果桶没有足够的空间，就会发生**桶溢出**，有以下几个可能原因：

- **桶不足**
- **偏斜**（某些桶分配到的记录比其它的桶多）

偏斜发生的原因：

- 多条记录可能具有相同的搜索码
- 所选的散列函数可能会造成搜索码的分布不均

我们用**溢出桶**来处理溢出问题，这种链接列表的溢出处理称为**溢出链**

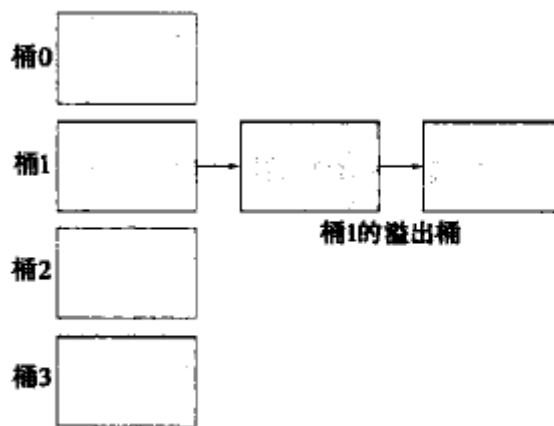


图 11-24 散列结构中的溢出链

上面这种形式的散列结构称为**闭地址**，下面这种方法称为**开地址**

桶集合是固定的，没有溢出链

当一个桶满了以后，系统将记录插入到其它桶中

该方法在数据库系统不使用，因为删除操作太麻烦

11.6.3 散列索引

散列索引将搜索码及其相应的指针组织成散列文件结构

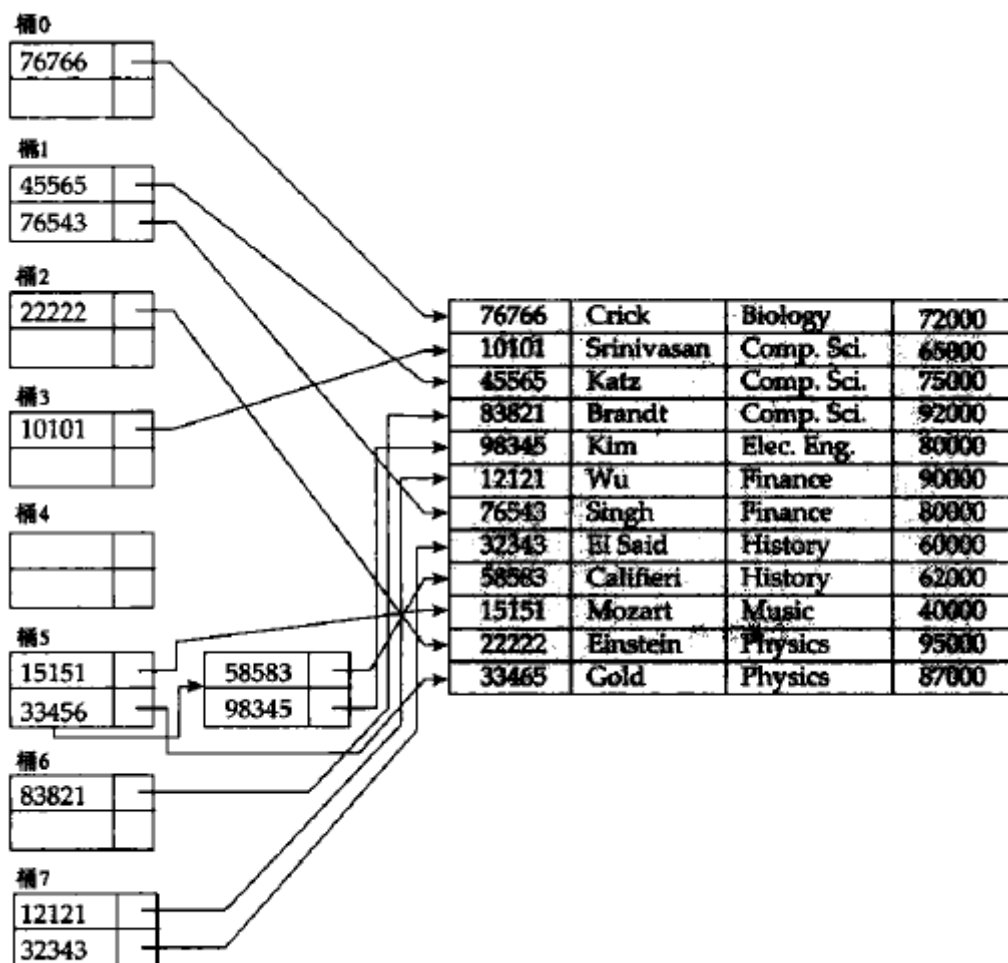


图 11-25 *instructor* 文件中在搜索码 ID 上的散列索引

11.7 动态散列

动态散列技术允许散列函数动态改变，以适应数据库增大或缩小的需要

11.7.1 数据结构

选择一个具有均匀性和随机性的散列函数 h ， h 可以产生值的范围较大，是 b 位二进制整数
开始时我们不使用 b 位，而是使用 i 位， i 会随着文件增长而变化

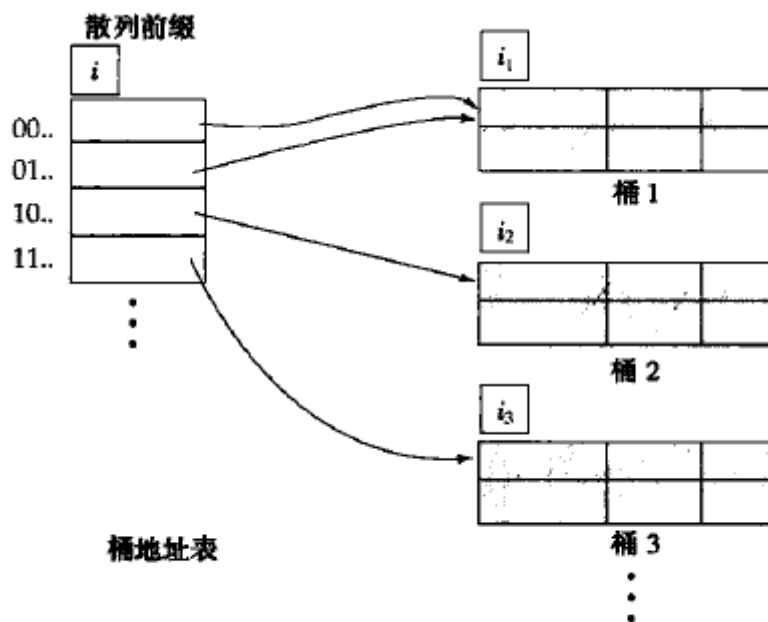


图 11-26 可扩充散列的一般结构

11.7.2 查询和更新

假定要插入一条搜索码值为 K 的记录，最终定位到某个桶 j

算法：

若桶 j 中有剩余空间，直接将记录插入

若桶 j 已经满，则分裂桶并重新分配记录

若 $i > i_j$ （前缀位数），那么在桶地址表中有多个表项指向桶 j ，直接分裂桶 j ，将 i 值设置为 $i_j + 1$ 即可

若 $i = i_j$ ，那么在桶地址表中只有一个表项指向桶 j ，增加桶地址表的大小，以容纳由于桶 j 分裂而产生的两个桶指针， $i = i + 1$ ，分裂出来的桶 i 值设置为 $i+1$ ，将原来桶中的记录重新分配

若分裂多次还不能成功（如搜索码值都是相同的情况），则采用溢出桶来记录

example:

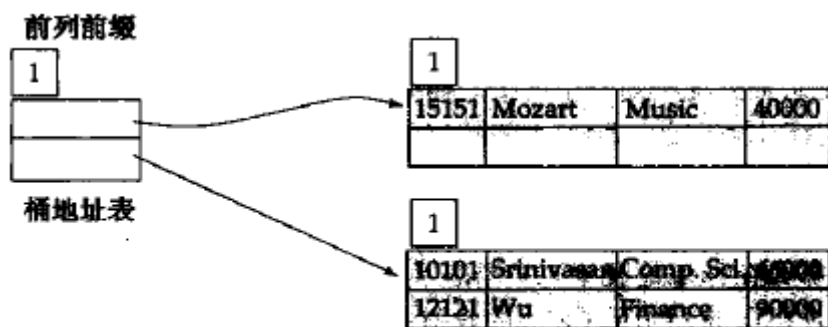


图 11-29 3 次插入操作后的散列结构

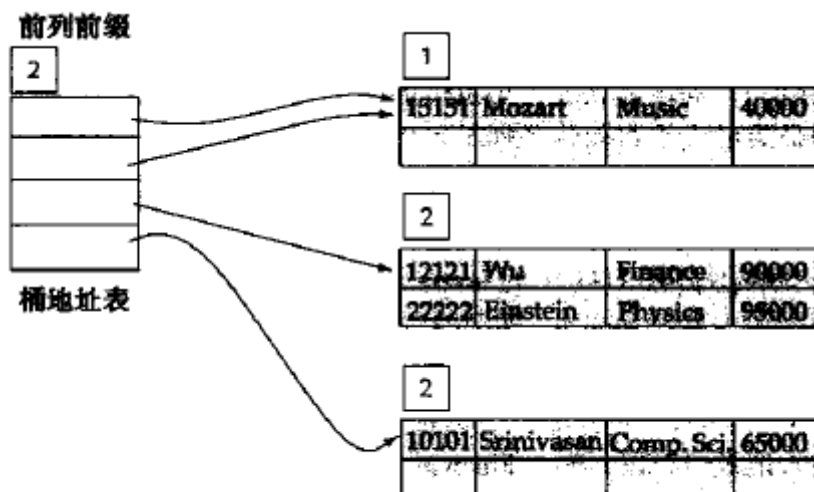


图 11-30 4 次插入操作后的散列结构

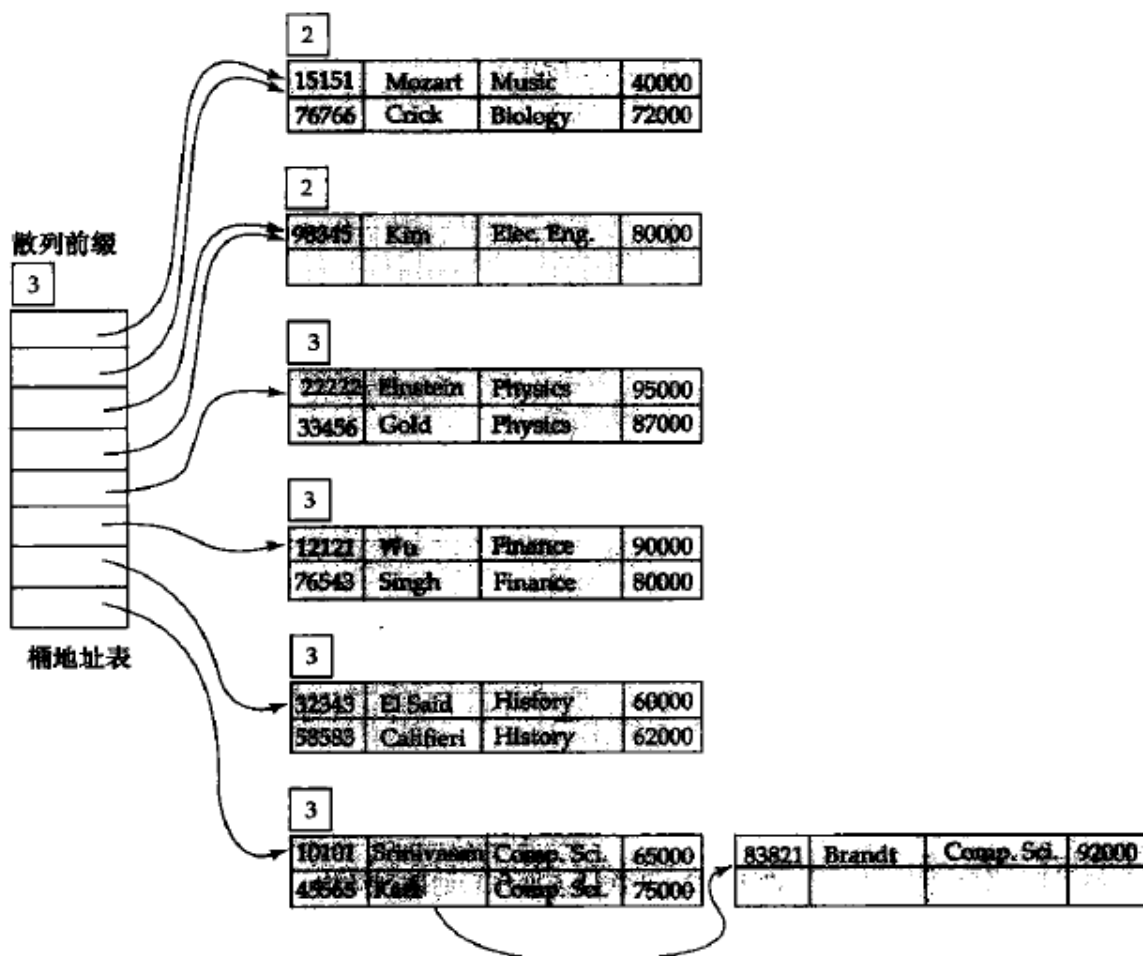


图 11-34 文件 *instructor* 的可扩充散列结构

11.7.3 静态散列与动态散列比较

可扩充散列

- 优点：
性能不随文件的增长而降低
空间开销是最小的，尽管桶地址表带来了额外的开销，但该表较小
- 缺点：
查找涉及一个附加的间接层，对性能有微小的影响

11.8 顺序索引和散列的比较

普通查询散列更有优势

```
select A1, A2, ..., An
from r
where Ai = c;
```

范围查询顺序索引更可取

```
select A1, A2, ..., An
from r
where A1 ≤ c1 and Ai ≥ ci
```

11.9 位图索引

位图索引是一种为多码上的简单查询设计的特殊索引，尽管每个位图索引都是建立在一个码之上的

11.9.1 位图索引结构

位图就是位的一个简单数组

关系r的属性A上的**位图索引**是由A能取的每个值建立的位图构成的

example:

gender: mffmf income_level: 12143

record number	Bitmaps for <i>gender</i>			Bitmaps for <i>income_level</i>	
	<i>ID</i>	<i>gender</i>	<i>income_level</i>		
0	76766	m	L1	m	10010
1	22222	f	L2	f	01101
2	12121	f	L1		
3	15151	m	L4		
4	58583	f	L3		
				L1	10100
				L2	01000
				L3	00001
				L4	00010
				L5	00000

查询 where gender = 'f' and income_level = 'L2'

我们执行两个位图的**交**操作，得到位图01000

统计满足条件的元组数时，甚至可以在不需要访问关系的条件下就可以从位图索引中得到需要的结构

11.11 总结

见P300